

Introduction to RxJava

Student Sheet

Introduction

In this exercise sheet we will go deeper into RxJava. In particular, we will work with:

- Combining and aggregating emissions
- Schedulers and concurrency
- Error handling
- Flow control

1 Exercise Statements

Module 1. Combining and aggregating emissions

Exercise 1. The merge

Use two separate observables, one that emits names and another that emits colors, and use **merge()** to combine them into a single sequence. Observe the order in which the values are emitted.

Tip: **merge()** interleaves values from multiple observables as they arrive, so it is useful when you want a single stream that reflects concurrent sources. For example: **Observable.merge(names, colors)**.

Exercise 2. Pairing emissions

Apply **zip()** to a sequence of names and another sequence of exam grades to emit **Grade** objects.

Tip: **zip()** waits until each source has produced one item, then combines those items into one result. It is the right choice when values belong together position by position, for example **Observable.zip(names, grades, Grade::new)**.

Exercise 3. Grouping

Given a list of words, use **groupBy** to group them by their first letter and count the number of words per letter.

Tip: **groupBy()** splits one observable into smaller observables by key. Here, each key is the first letter, and each group can be counted separately with **count()**.

Exercise 4. Reduction

Use **reduce()** to compute the sum of a sequence of integers.

Tip: **reduce()** accumulates all the emitted items into a single final value. In a sum, the accumulator starts at 0 and adds each new number to the running total.

Module 2. Schedulers and concurrency

Exercise 5. Thread switch

Create an observable that performs a heavy computation, which can be simulated with a delay of about 2 seconds. Use `subscribeOn()` to run it in the background and `observeOn()` to display the result on another thread.

Tip: `subscribeOn()` decides where the upstream work starts, while `observeOn()` changes the thread used from that point onward for downstream operators and the subscriber.

Exercise 6. Parallel challenge

The code in this exercise simulates the process of fetching 20 resources from the Internet. These tasks are launched through another observable with a lambda function, and each request is supposed to take half a second.

Check that this is a sequential solution and, therefore, the total execution time will be about 10 seconds. Make the necessary changes to get a parallel solution, that is, make the tasks run on a thread separate from the “main” thread. Check the time difference with respect to the initial solution.

Change `subscribe()` to `blockingSubscribe()` and analyze the differences.

Tip: A good way to parallelize independent work is to use `flatMap()` so each request can run independently, and place the asynchronous work on a background scheduler.

Module 3. Error handling

Exercise 7. Retry after a failure

Create an observable that simulates an operation that fails frequently, for example, with probability 0.7 it throws a `RuntimeException`. Use `retry()` to allow the operation to be retried 3 times before failing definitively.

Tip: `retry()` re-subscribes automatically when an error happens. With a count, it gives the source a limited number of new chances before forwarding the failure.

Exercise 8. A plan B

Create an observable that emits three integer values, but with probability 0.5 it throws a `RuntimeException` instead of the corresponding value. Use `onErrorResumeNext()` to provide an observable that emits values 10, 11, and 12 in case of error.

Tip: `onErrorResumeNext()` replaces a failing stream with another one, so you can keep the pipeline alive by switching to a fallback observable.

Module 4. Flow control

Exercise 9. Buffer

Create an observable that simulates mouse clicks every 100 ms with `interval()`. Group the events into one-second windows with `buffer()` to process them at that rate. For each group, show the number of events.

Tip: `buffer()` collects items into lists before emitting them, which is useful when you want to process bursts of events in batches instead of one by one.

Exercise 10. Window

Do the same as in the previous exercise, but using `window()`.

Tip: `window()` also groups items over time, but it emits each group as another observable instead of turning it into a list. That makes it useful when each batch must still be processed reactively.

Exercise 11. Debounce

Start from the `keyStrokes` observable that simulates a user typing characters on the keyboard with variable speed. Use `debounce()` to discard characters that are followed by another one before 50 ms, because they are considered artificial keystrokes due to being too fast.

Tip: `debounce()` waits for silence before letting a value through, so it filters out fast intermediate keystrokes and keeps only the last one after a pause.

Exercise 12. Sample

Start again from the `keyStrokes` observable. Now use `sample()` to sample events every 50 ms, because the typing speed is considered to be constant at that pace. Compare the result of this exercise with the previous one.

Tip: `sample()` periodically takes the latest value seen so far, which is useful when you want snapshots at a fixed rate instead of waiting for pauses.

Exercise 13. Excessive pressure

Start from the `obs` observable, which emits an object of about 1 KB every nanosecond. Subscribe a slow subscriber that needs 100 ns for each emission. Observe that everything runs on the same thread and, therefore, the observable has to wait for the subscriber.

Now move the subscriber to another thread with `observeOn()`. The observable can emit at the indicated rate, but the subscriber cannot keep up. Try changing the emission or subscription speed to see the effect.

Tip: `observeOn()` is what moves the slow consumer work onto another thread, which makes the effect of backpressure visible when the producer is faster than the subscriber.

Exercise 14. Pressure control

We are going to repeat the previous exercise, but replacing `Observable` with `Flowable`. An exception is produced again, in this case because no backpressure strategy has been defined. Try introducing backpressure strategies with `onBackpressureDrop()` or `onBackpressureLatest()` and analyze the behavior.

Tip: `Flowable` adds backpressure support, and operators like `onBackpressureDrop()` or `onBackpressureLatest()` decide what to do when the consumer cannot keep up.